

Design Patterns

Guía explicativa con analogías, casos de uso y ejemplos en JavaScript, pensada para repasar los patrones creacionales, estructurales y de comportamiento vistos en clase.

1. ¿Qué es un patrón de diseño?
2. Patrones Creacionales — Singleton, Factory
3. Patrones Estructurales — Composite, Decorator
4. Patrones de Comportamiento — State, Strategy, Observer
5. Tabla comparativa y cuándo usar cada uno

INTRODUCCIÓN

¿Qué es un Design Pattern?

Un patrón de diseño **no es código que se copia y pega**: es una solución general y reutilizable a un problema que aparece una y otra vez al diseñar software. Pensalo como una "receta" probada — te dice qué forma darle a las clases y objetos para resolver un problema típico, pero la implementación concreta depende de cada caso.

Analogía: es como un plano de arquitectura estándar (por ejemplo, "cómo construir una escalera segura"). No te da los planos exactos de tu casa, pero sí el criterio para que, cada vez que necesites una escalera, no tengas que reinventar la solución desde cero.

Los patrones se agrupan en tres familias según el problema que atacan:

Familia	Pregunta que responde	Patrones de esta guía
Creacionales	¿Cómo creo objetos de forma flexible?	Singleton, Factory
Estructurales	¿Cómo compongo clases/objetos en estructuras más grandes?	Composite, Decorator
De comportamiento	¿Cómo se comunican y reparten responsabilidades los objetos?	State, Strategy, Observer

PATRONES CREACIONALES

Singleton CREACIONAL

Idea central: garantizar que una clase tenga **una única instancia** en toda la aplicación, y dar un punto de acceso global a ella.

Analogía: pensá en la *presidencia de un país*. No importa cuántas veces alguien "pida" al presidente, siempre te van a devolver la misma persona — no se crea uno nuevo cada vez que alguien lo necesita.

¿Cuándo usarlo?

1. Cuando debe existir una sola instancia de una clase y necesitás que sea accesible desde cualquier parte del código (ej: una conexión a base de datos, un logger, una configuración global).
2. Cuando esa instancia única necesita poder extenderse por subclases sin que el código cliente tenga que cambiar.

Puntos clave

- Acceso controlado y centralizado a la única instancia.
- Evita ensuciar el espacio de nombres global con variables sueltas.
- Permite optimizar recursos (por ejemplo, no abrir 10 conexiones cuando con una alcanza).
- Con algunas variantes, incluso se puede permitir un número controlado de instancias.

Ejemplo — con clases

La clave está en el **constructor**: si ya existe una instancia guardada en `Singleton.instance`, la devuelve en lugar de crear una nueva.

```
class Singleton {
  constructor() {
    if (Singleton.instance) {
      return Singleton.instance; // ya existe: la devuelvo
    }
    this.value = Math.random();
    Singleton.instance = this; // primera vez: la guardo
  }
  getValue() {
    return this.value;
  }
}

const singletonA = new Singleton();
const singletonB = new Singleton();
console.log(singletonA === singletonB); // true -> son EL MISMO objeto
```

Ejemplo — con closures (prototipos)

Misma idea, pero usando una función que "recuerda" (closure) la variable `instance` entre llamadas:

```
const Singleton = (function() {
  let instance;
  return function() {
    if (!instance) instance = {};
    return instance;
  };
})();

const a = Singleton();
const b = Singleton();
console.log(a === b); // true
```

Factory

CREACIONAL

Idea central: en vez de instanciar objetos directamente con `new` por todos lados, se centraliza esa creación en una función (o método) que decide **qué clase concreta crear** según algún parámetro.

Analogía: es como pedir comida en un *restaurante*. Vos solo decís "quiero una hamburguesa" (el pedido/rol), y la cocina (la Factory) se encarga de saber exactamente qué ingredientes y proceso usar para construirla. No necesitás saber la receta interna para pedir.

¿Cuándo usarlo?

1. Cuando una clase no puede anticipar de antemano qué tipo de objeto va a necesitar crear.
2. Cuando querés que sean las subclases (o un parámetro) las que decidan qué objeto concreto se instancia.
3. Cuando hay varias "familias" de objetos similares y querés centralizar el conocimiento de cuál usar.

Puntos clave

- El código que *pide* el objeto no necesita conocer la clase concreta que se crea.
- Facilita agregar nuevos tipos: solo se toca la Factory, no todo el código que la usa.
- Evita esparcir `if/switch` de creación de objetos por toda la app.

Ejemplo

Tres "roles" distintos (`Admin` , `Guest` , `Member`) comparten la misma interfaz (`sayHello`), y una única función decide cuál instanciar:

```
function Admin(name) { this.role = 'admin'; this.name = name; }
Admin.prototype.sayHello = function() {
  return `Hola, soy ${this.name} y soy administrador.`;
};

function Guest(name) { this.role = 'guest'; this.name = name; }
Guest.prototype.sayHello = function() {
  return `Hola, soy ${this.name} y soy invitado.`;
};

function Member(name) { this.role = 'member'; this.name = name; }
Member.prototype.sayHello = function() {
  return `Hola, soy ${this.name} y soy miembro.`;
};

// La Factory: centraliza la decisión de QUÉ crear
function UserFactory(role, name) {
  switch (role) {
    case 'admin': return new Admin(name);
    case 'guest': return new Guest(name);
    case 'member': return new Member(name);
    default: throw new Error('Rol no válido');
  }
}

var user1 = UserFactory('admin', 'Ana');
console.log(user1.sayHello()); // Hola, soy Ana y soy administrador.
```

PATRONES ESTRUCTURALES

Composite

ESTRUCTURAL

Idea central: organizar objetos en una estructura de **árbol** (parte-todo) para que el código cliente pueda tratar a un objeto individual (hoja) y a un grupo de objetos (composición) **de la misma manera**.

Analogía: pensá en el *sistema de carpetas de tu computadora*. Una carpeta puede contener archivos sueltos o más carpetas (que a su vez contienen más archivos o carpetas). Si le pedís "calculame el tamaño total", no importa si es un archivo simple o una carpeta con 500 archivos adentro: la operación se aplica igual, de forma recursiva.

¿Cuándo usarlo?

1. Cuando necesitás representar jerarquías tipo árbol de objetos (partes y todos).
2. Cuando querés que el código cliente no tenga que distinguir entre "un objeto simple" y "un grupo de objetos" — que los trate igual.

Puntos clave

- Simplifica mucho el código cliente: no hace falta un `if` para preguntar "¿sos una hoja o un grupo?".
- Agregar nuevos tipos de componentes es fácil, ya funcionan con el código existente.
- **Contra:** el diseño puede volverse demasiado genérico — es difícil restringir qué tipos de hijos puede tener un composite (hay que chequearlo en tiempo de ejecución).

Ejemplo

Un `Leaf` (hoja) solo sabe mostrarse a sí mismo. Un `Composite` puede contener hijos (hojas u otros composites) y, al mostrarse, delega la misma operación en cada uno de ellos:

```
function createLeaf() {
  return { show: () => console.log("Leaf") };
}

function createComposite() {
  const children = [];
  return {
    add: c => children.push(c),
    show: () => {
      console.log("Composite");
      children.forEach(child => child.show()); // misma llamada, sea hoja o grupo
    }
  };
}

const root = createComposite();
root.add(createLeaf());
root.add(createLeaf());
root.show();
// Composite
// Leaf
// Leaf
```

Decorator ESTRUCTURAL

Idea central: agregar funcionalidad extra a un objeto **de forma dinámica**, "envolviéndolo" en otro objeto que respeta la misma interfaz, en lugar de crear una subclase para cada combinación posible de funcionalidades.

Analogía: es literalmente pedir un *café*: empezás con un café base y le vas "decorando" capas encima — leche, azúcar, canela — sin necesitar una clase distinta para cada combinación posible (CaféConLeche, CaféConAzúcar, CaféConLecheYAzúcar...). Cada decorador simplemente envuelve al anterior y le suma algo.

¿Cuándo usarlo?

1. Cuando querés agregar (o quitar) responsabilidades a objetos puntuales, sin afectar a otros.
2. Cuando usar herencia generaría una explosión combinatoria de subclases (una por cada combinación de funcionalidades).

Puntos clave

- Más flexible que la herencia: se pueden agregar/quitar comportamientos en tiempo de ejecución.
- Evita clases gigantes que intentan prever todas las combinaciones posibles.
- **Cuidado:** un objeto decorado deja de ser "idéntico" (===) al original — no confiar en identidad.
- Genera muchos objetos pequeños y parecidos entre sí; puede ser difícil de depurar si no se conoce el patrón.

Ejemplo

Cada decorador envuelve al café anterior y le suma costo y descripción:

```
function coffee() {
  return { cost: () => 5, description: () => 'Café' };
}
function milkDecorator(coffeeObj) {
  return {
    cost: () => coffeeObj.cost() + 2,
    description: () => coffeeObj.description() + ' + leche'
  };
}
function sugarDecorator(coffeeObj) {
  return {
    cost: () => coffeeObj.cost() + 1,
    description: () => coffeeObj.description() + ' + azúcar'
  };
}

let c = coffee();
let milkSugarCoffee = sugarDecorator(milkDecorator(c));
console.log(milkSugarCoffee.cost()); // 8
console.log(milkSugarCoffee.description()); // 'Café + leche + azúcar'
```

PATRONES DE COMPORTAMIENTO

State

COMPORTAMIENTO

Idea central: permitir que un objeto cambie su comportamiento cuando cambia su estado interno, como si en cada momento "cambiara de clase".

Analogía: pensá en un *semáforo*. El mismo objeto ("semáforo") se comporta distinto según su estado: en rojo hace una cosa, en verde otra, en amarillo otra. En vez de tener un gran `switch` repetido en cada método, cada estado (rojo/verde/amarillo) es su propio objeto que sabe cómo comportarse y a qué estado pasar después.

¿Cuándo usarlo?

1. Cuando el comportamiento de un objeto depende fuertemente de su estado y debe cambiar en tiempo de ejecución.
2. Cuando tenés `if/switch` larguísimos repetidos en varios métodos, todos preguntando por el mismo estado.

Puntos clave

- Cada estado concentra su propio comportamiento en un objeto separado — fácil agregar estados nuevos.
- Hace explícitas las transiciones de estado (en vez de "variables mágicas" internas).
- Si los objetos de estado no tienen datos propios, se pueden compartir entre varios contextos.

Ejemplo

Un interruptor que alterna entre estado "ON" y "OFF"; cada estado sabe qué hacer al recibir `handle()`:

```
const onState = { handle: () => console.log("ON") };
const offState = { handle: () => console.log("OFF") };

function createSwitch() {
  let state = offState;
  return {
    press: function () {
      state.handle();
      state = (state === offState) ? onState : offState; // transición explícita
    }
  };
};

const s = createSwitch();
s.press(); // OFF
s.press(); // ON
```

Strategy COMPORTAMIENTO

Idea central: encapsular una familia de algoritmos intercambiables, de forma que el algoritmo pueda variar de forma independiente de quien lo usa.

Analogía: es como elegir *cómo viajar a algún lugar*: podés ir en auto, en bici o caminando. El objetivo (llegar a destino) es el mismo, y vos (el cliente) simplemente elegís qué "estrategia" de viaje usar según el contexto, sin que eso cambie la lógica de "quiero llegar a destino".

¿Cuándo usarlo?

1. Cuando tenés varias clases que se parecen mucho y solo difieren en su comportamiento.
2. Cuando una clase tiene muchos `if/switch` para elegir entre distintos comportamientos — cada rama puede convertirse en una Strategy separada.
3. Cuando necesitás distintas variantes de un mismo algoritmo (por ejemplo, distintos balances entre velocidad y uso de memoria).

Puntos clave

- Elimina condicionales largos: el algoritmo se selecciona, no se "if-easea".
- El algoritmo puede cambiarse en tiempo de ejecución sin tocar el contexto que lo usa.
- **Contra:** el cliente necesita saber qué estrategias existen para poder elegir la correcta.

Ejemplo

El "contexto" no sabe cómo se calcula el resultado — solo delega en la estrategia activa:

```
const add      = { execute: arr => arr.reduce((a,b) => a+b, 0) };
const multiply = { execute: arr => arr.reduce((a,b) => a*b, 1) };

function createContext(strategy) {
  let strat = strategy;
  return {
    setStrategy: s => strat = s,
    execute: arr => strat.execute(arr)
  };
}

const c = createContext(add);
console.log(c.execute([1,2,3])); // 6
c.setStrategy(multiply);
console.log(c.execute([1,2,3])); // 6 (2*3*1, distinto algoritmo, misma interfaz)
```

Observer

COMPORTAMIENTO

Idea central: definir una relación de dependencia uno-a-muchos, de modo que cuando un objeto ("sujeto") cambia de estado, todos sus "observadores" son notificados y actualizados automáticamente.

Analogía: es como *suscribirse a un canal de YouTube*. El canal (sujeto) no sabe ni le importa quiénes son sus suscriptores exactamente; cuando sube un video nuevo, simplemente notifica a todos los que estén suscritos (observadores), sin acoplarse a cada uno individualmente.

¿Cuándo usarlo?

1. Cuando un cambio en un objeto requiere actualizar a otros, y no sabés (ni querés fijar de antemano) cuántos ni cuáles son esos otros objetos.
2. Cuando querés que un objeto pueda "avisar" a otros sin acoplarse fuertemente a ellos.

Puntos clave

- Acoplamiento mínimo: el sujeto solo conoce la interfaz genérica de "observador".
- Soporta broadcasting: notifica a todos los interesados sin especificar destinatarios.
- **Cuidado:** una notificación puede disparar una cascada de actualizaciones inesperada, ya que los observadores no se conocen entre sí.

Ejemplo

El `subject` mantiene una lista de observadores y los notifica a todos por igual:

```
function createSubject() {
  const observers = [];
  return {
    attach: obs => observers.push(obs),
    notify: msg => observers.forEach(o => o.update(msg))
  };
}

function createObserver() {
  return { update: msg => console.log("Received:", msg) };
}

const s = createSubject();
const o1 = createObserver(), o2 = createObserver();
s.attach(o1);
s.attach(o2);
s.notify("Hello"); // ambos observadores reciben "Hello"
```

PARA CERRAR

Tabla comparativa

Patrón	Resuelve...	Se usa cuando...
Singleton	Que exista una sola instancia	Necesitás un único punto de acceso global (config, conexión, logger)
Factory	Crear objetos sin acoplarse a la clase concreta	Hay varias clases relacionadas y el "cuál crear" depende de un dato en runtime
Composite	Tratar grupos e individuos por igual	Tenés una jerarquía parte-todo tipo árbol
Decorator	Agregar funcionalidad sin herencia	Necesitás combinar responsabilidades dinámicamente

State	Cambiar comportamiento según estado	Tenés condicionales largos basados en el estado del objeto
Strategy	Intercambiar algoritmos	Hay varias formas de resolver lo mismo y querés poder elegir/cambiar
Observer	Notificar cambios a muchos interesados	Un cambio de estado debe propagarse a N objetos desacoplados

Basado en el material de clase "Design Patterns" (Programación Orientada a Objetos 2) — referencias: Freeman & Robson, *Head First Design Patterns* (O'Reilly, 2020); Gamma, Helm, Johnson & Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software* (1994).